



Yesterday 7:35 AM

I have an use case how to automate it using gemini or claude subscription

I have a WhatsApp group where group of friends learning together on various topic. Want to make a help document out of that WhatsApp group chat, it should be dynamic, connected and update the document daily. It can list down the content to be updated, fine tune and ask for the confirmation to store. I need a workflow custom solution for this



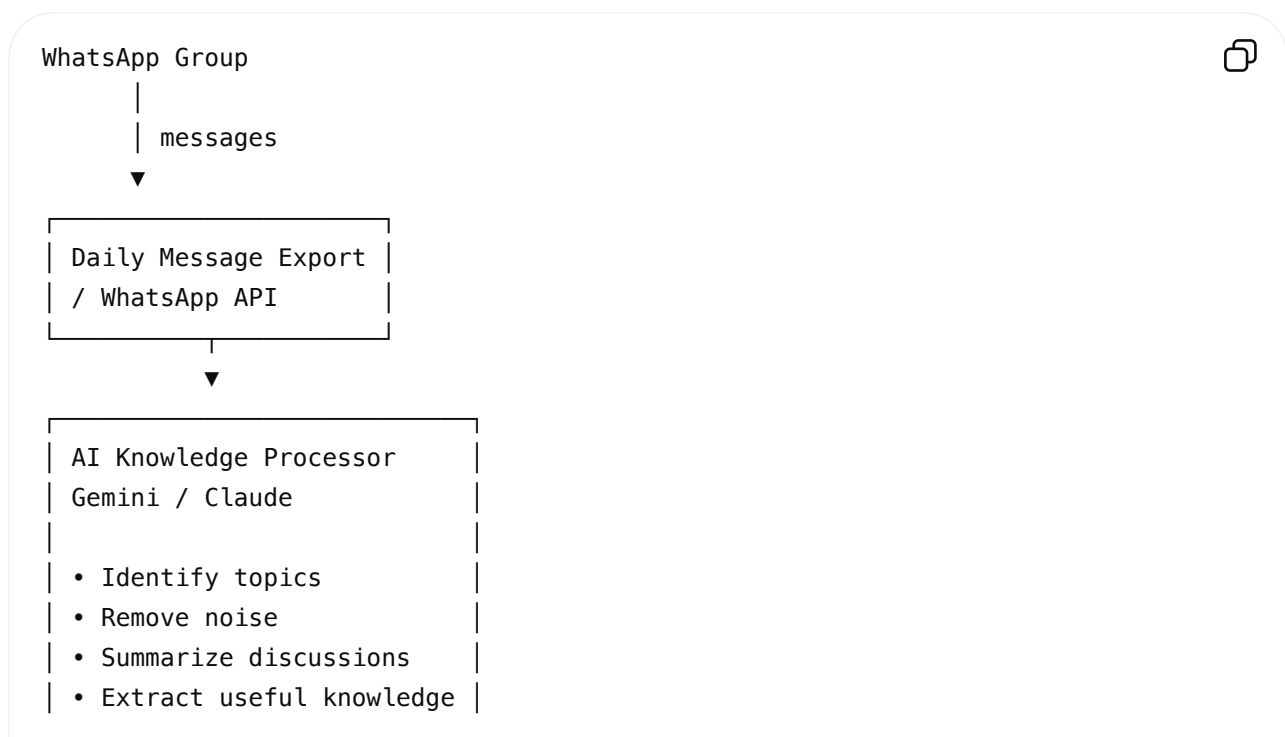
Yes. This is a very good use case for an **AI-powered living knowledge base**. You don't need to manually copy WhatsApp messages into Gemini/Claude every day.

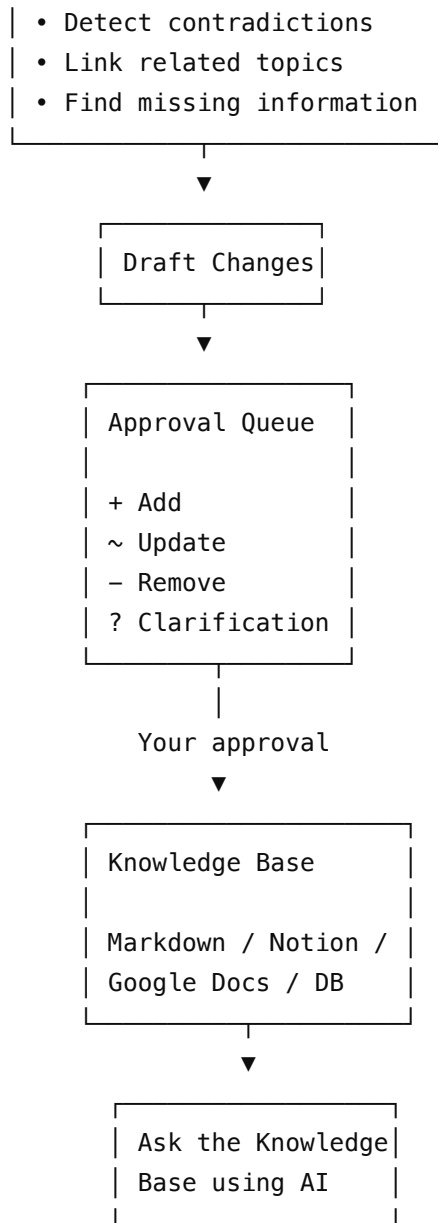
The architecture I'd recommend is:

WhatsApp Group → Daily Chat Capture → AI Processing → Proposed Updates → Your Approval → Knowledge Document → Search/Ask AI

The important part is separating **AI suggestions** from **approved knowledge**.

A practical custom workflow





I'd actually build it slightly differently

Don't make **one giant document** the source of truth.

Instead:

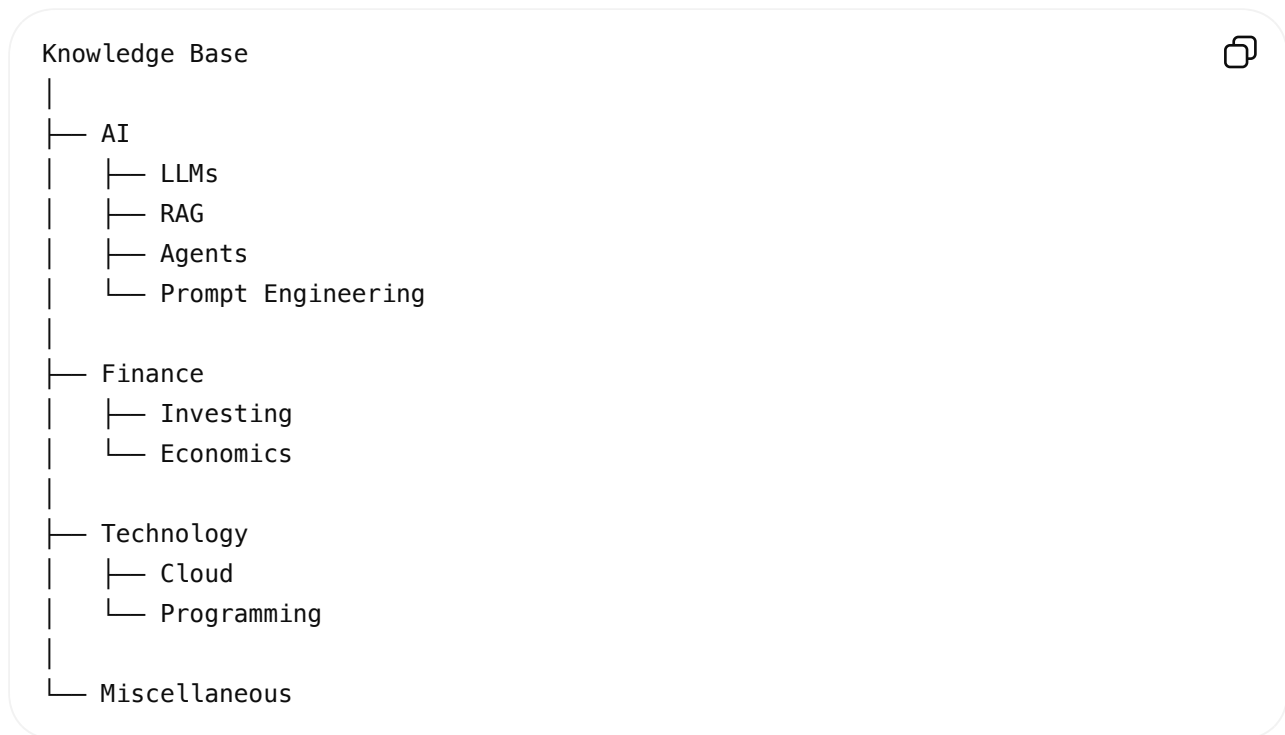
1. Raw WhatsApp archive

Keep the original messages with:

- date/time
- sender
- message
- reply/thread information if available
- links
- attachments

2. Structured knowledge base

For example:



Each topic becomes its own knowledge item.

3. Human approval layer

Every day the AI generates something like:

Daily Knowledge Update — 10 Sep

3 proposed changes

1. Add — RAG

The group discussed why vector databases are used in RAG systems...

Source: 14 messages from 9 Sep

2. Update — LLM Context Window

Existing explanation says X. Today's discussion suggests adding Y.

3. Possible contradiction

Two members gave different explanations of...

Needs confirmation

[Approve all] [Review individually] [Reject]

Only after approval does it modify the actual knowledge base.

Gemini vs Claude

For this particular workflow, **both can work**, but I would not make the subscription itself the architecture.

I'd build an application around the API of whichever model you choose.

Claude

Very attractive if the workflow is heavily document-oriented and you want excellent reasoning over a growing knowledge base.

Gemini

Very attractive if you're already using Google Workspace — especially if the final knowledge system lives in **Google Drive / Docs / Sheets**.

My preference for your use case

If your friends already use Google:

WhatsApp → Gemini API → Google Drive/Docs + database

would be a very clean solution.

If you want a more sophisticated knowledge system:

WhatsApp → Claude API → Markdown/DB → AI interface

is also excellent.

And importantly, **your Claude/Gemini consumer subscription is generally not the same thing as API access**. For an automated system, we'd normally use the provider's API and pay based on usage.

The really interesting part: make it "connected"

This is where your idea becomes much better than an automated summary.

Suppose Monday's discussion is:

"What is RAG?"

Tuesday:

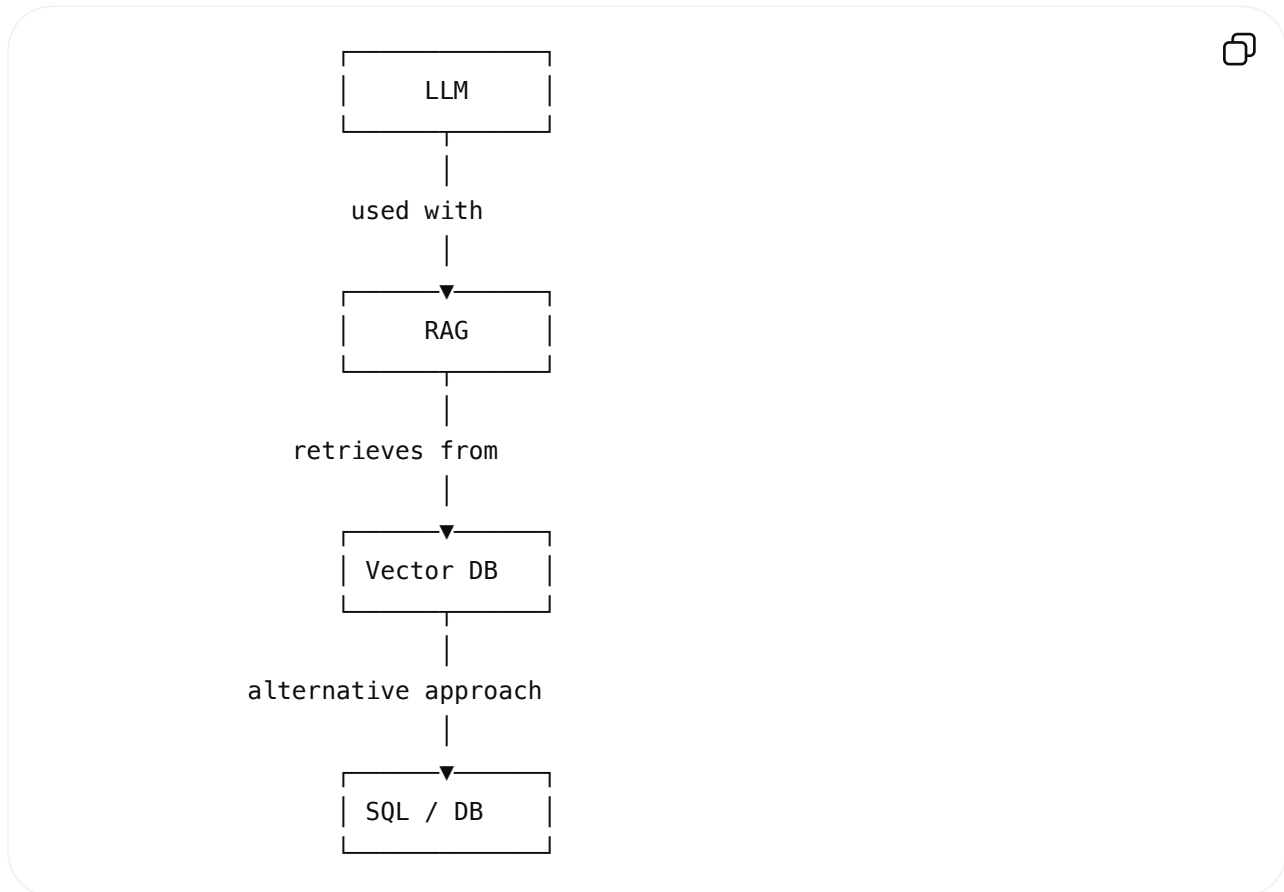
"Why do we need vector databases?"

Wednesday:

"Can an LLM use SQL instead of vectors?"

The AI shouldn't create three disconnected pages.

It should understand:



So each knowledge item can contain:

Concept

Explanation

Examples

Related concepts

Questions

Open issues

Sources

WhatsApp discussion references

Last updated

Confidence

This gives you a real **knowledge graph**, not merely a document.

Daily automation

I'd run this once every night.

For example:

11:00 PM

Get yesterday's WhatsApp messages

↓

Identify meaningful discussions

↓

Group messages by topic

↓

Compare with existing knowledge

↓

Generate proposed changes

↓

Check for contradictions

↓

Generate approval report

↓

Send report to you



Then you could receive:



Knowledge Base Update

Today's discussion produced **5 potential updates**



2 new concepts



2 improvements



1 conflicting explanation

Review:

[Review Updates]

You approve them.

Then:

Approved



AI generates clean knowledge



Update database



Regenerate topic document



Create version history



One important decision: where should the final knowledge live?

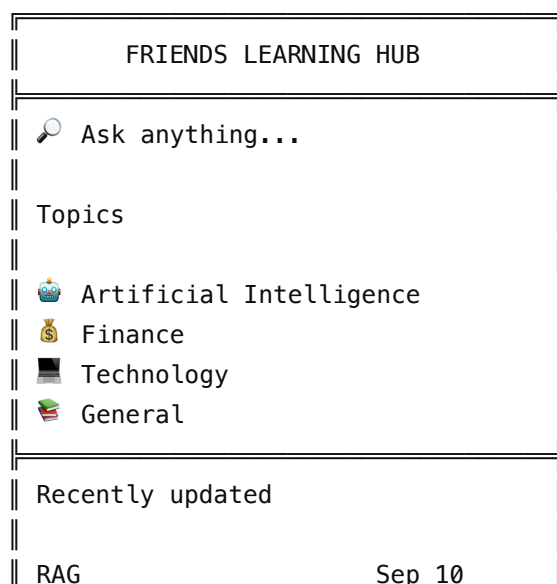
I'd consider these four:

Destination	Good for
Notion	Best friend-facing knowledge wiki
Google Docs/Drive	Simple, familiar
GitHub + Markdown	Version controlled, technical
Custom DB + web UI	Best long-term solution

For your group, I'd probably start with:

Notion or Google Drive + a small database behind it.

Later you can build a web interface:



LLM Context Windows	Sep 9
AI Agents	Sep 8

And people can ask:

"What did we learn about RAG?"

or

"Connect everything we've discussed about AI agents."

The AI answers **from your group's accumulated knowledge**, while showing the original WhatsApp discussions as provenance.

The biggest technical challenge

Getting WhatsApp messages automatically.

This is the part I'd solve first.

There are essentially three approaches:

A. WhatsApp export → automation

Simplest MVP.

Someone periodically exports the group chat and puts the file into a folder.

The system processes it automatically.

Very easy to build, but not truly daily/automatic.

B. WhatsApp Business/API route

Much more automated, but WhatsApp's official APIs have restrictions and are primarily designed around business messaging rather than arbitrarily reading personal/group chats.

C. WhatsApp Web automation

Technically possible to automate WhatsApp Web, but I **wouldn't make this the foundation of a production system**. It can be brittle and may conflict with WhatsApp's terms/anti-automation mechanisms.

What I would build for you

I'd make this a **small custom application**, not a complicated enterprise system.

Version 1

```
WhatsApp exported chat
  ↓
  Python
  ↓
  Gemini / Claude API
  ↓
  SQLite/Postgres
  ↓
  Proposed Changes
  ↓
  Approval dashboard
  ↓
  Markdown knowledge base
```



Then Version 2:

```
WhatsApp
  ↓
  Automated ingestion
  ↓
  AI processing
  ↓
  Knowledge graph
  ↓
  Approval
  ↓
  Notion / Google Drive
  ↓
  RAG chatbot
```



And I'd make the AI prompt/schema very strict

For example, the AI doesn't get permission to directly modify knowledge.

It must return:

```
</> JSON

{
  "topic": "RAG",
  "changes": [
    {
      "type": "UPDATE",
```



```
    "section": "Why RAG is needed",
    "proposed_change": "...",
    "reason": "...",
    "source_messages": ["msg_123", "msg_127"],
    "confidence": 0.91
  }
],
"questions_for_human": [],
"related_topics": [
  "Vector Databases",
  "Embeddings",
  "LLMs"
]
}
```

Your approval is the **gate** between AI and the permanent knowledge base.

That makes the system much safer and much easier to maintain.

If you want to actually build this

I can take this from concept to a **concrete implementation architecture**, including:

1. WhatsApp ingestion method
2. Gemini vs Claude choice
3. Database schema
4. AI prompts
5. Knowledge-document structure
6. Daily automation
7. Approval UI
8. Notion/Google Docs integration
9. RAG/search architecture
10. Actual Python/Node code for the MVP
11. Estimated API cost for, say, a 50/100/500-message-per-day group

And we can design it so that **you can build and run the first version yourself without needing a full software team.**



I checked the names against current web results. One caveat: **web availability ≠ trademark/domain availability**. I can identify names that already have clear commercial use, but a proper trademark and domain check is still needed before you commit.

✖ Names I'd eliminate

Name	Why
LearnLoop	Already an established education/AI product, LearnLoop Labs.  LearnLoop
MindMesh	Already an AI productivity/workspace product, plus other projects using the name.  MINDMESH +1
KnowFlow	Already an AI knowledge-base product.  KnowFlow
LearnGraph	Already an established learning/knowledge-graph platform, including LearnGraph.dev.  LearnGraph +1
CollectiveIQ	Already used by a consulting company.  CollectiveIQ
LearnOS	Multiple existing education products use it.  LearnOS +2
NexMind	Established AI SEO company.  Nexmind
Cognify	Existing data-science/AI company.  Cognify
KnowledgeOS	Already an active AI knowledge-base product and open-source project.  KnowXWorks +1
Cortex	Extremely crowded — multiple AI/knowledge products already use it.  Cortex +2
Synapse	Very heavily used in technology/AI/research.  Synapse +1

🟡 Names with weaker/unclear conflicts

From the searches I ran, **BrainSync, WisdomLoop, MindVault, ChatterMind and ThinkFlow** did not surface an obvious dominant company that makes them immediately unusable.

However, I would **not yet call these "available."** We'd need to check trademarks and domains.

But I'd actually go in a different direction

Your concept is sufficiently interesting that I wouldn't settle for a generic AI name.

The essence is:

A conversation that keeps learning and becomes a shared, evolving brain.

So I'd look for a name that communicates **conversation → knowledge → connections → continuous learning.**

My new shortlist

1. MindWeave ★★★★★

Weaving everyone's conversations into collective knowledge.

2. KnowWeave ★★★★★

Knowledge continuously woven from discussions.

3. LearnWeave ★★★★★

Collective learning woven together.

4. ThinkWeave ★★★★★

Ideas from different people become one connected knowledge system.

5. WisdomMesh ★★★★★

Collective wisdom connected into a mesh.

6. MindLedger ★★★★★

A continuously updated record of what the group has learned.

7. LearnLedger ★★★★★

Every useful conversation becomes part of the group's learning history.

8. IdeaForge ★★★★★

Raw discussions are forged into useful knowledge.

9. KnowledgeForge ★★★★★

Strong meaning, though more generic.

10. BrainWeave ★★★★★

Probably the closest metaphor to what you're actually building.

My top 3 for *your* project

🏆 1 MindWeave

🥈 2 ThinkWeave

🥉 3 BrainWeave

And I particularly like:

MindWeave

Conversations that become collective knowledge.

It leaves room for the project to grow beyond WhatsApp. Today it's WhatsApp conversations; tomorrow it could ingest **Telegram, Slack, meetings, PDFs, articles, YouTube transcripts, etc.**

If you're seriously thinking of eventually making this a **real product**, I can do a deeper **name-screening exercise** next: check these 10–15 candidates across **Google/web, GitHub, .com/.ai/.io domains, Indian company/trademark databases, and existing AI products**, then give you a **ranked list of names with the best chance of being genuinely ownable**.



LearnWeave, LearnLedger, MindWeave, BrainWeave
pick any one out of this

MindWeave 🏆

It's the strongest of the four.

Why:

- **Broad** — not limited to “learning”; can evolve into collective knowledge.
- **Memorable** — easy to say and spell.
- **Fits the concept** — multiple people's ideas and conversations being *woven together*.
- **Product-friendly** — sounds like something that could become a real AI product.
- **Future-proof** — works even if you later add documents, videos, meetings, Slack, etc.

MindWeave

Tagline: *Where conversations become collective knowledge.*

That's the one I'd choose.



i am going to develop this using codex, how to refer this chat over there ?

Yes. Since you're going to build **MindWeave** in Codex, the cleanest approach is to give Codex a **project brief distilled from this conversation**, rather than expecting it to access this chat automatically.

You can start your Codex session with something like:

MindWeave — Codex Project Brief



MindWeave

Vision

MindWeave is an AI-powered collective learning system that converts conversations from a private learning group into a continuously evolving, connected knowledge base.

Tagline: Where conversations become collective knowledge.

Core Use Case

A group of friends uses a WhatsApp group to learn and discuss different topics.

The system should:

1. Ingest WhatsApp group conversations.
2. Process new messages daily.
3. Identify meaningful learning content.
4. Ignore casual/noise messages.
5. Group discussions into topics.
6. Extract concepts, explanations, examples, questions and insights.
7. Compare new discussions against existing knowledge.
8. Identify:
 - New knowledge
 - Improvements to existing knowledge
 - Contradictions
 - Duplicate information
 - Unresolved questions
 - Related topics
9. Generate proposed knowledge-base changes.
10. Ask for human confirmation before permanently updating the knowledge base.
11. Store approved changes.
12. Maintain history/versioning and source references back to the original conversation.
13. Keep the knowledge interconnected rather than creating isolated summaries.
14. Allow users to ask questions against the accumulated knowledge.

Important Design Principle

The AI must NOT directly modify the permanent knowledge base.

Use this pipeline:

WhatsApp

- Raw conversation store
- AI analysis
- Proposed changes
- Human approval
- Approved knowledge
- Connected knowledge base
- AI Q&A

Initial Architecture

Start with an MVP that is simple enough to develop and test locally.

Suggested architecture:

- Backend: Python
- AI: Gemini API or Claude API, designed so the model provider can be swapped
- Database: SQLite initially, with PostgreSQL as a future option
- Knowledge representation: structured records + Markdown/document representation
- Web UI: lightweight approval dashboard
- Daily processing: scheduled job
- Source messages: preserve original message IDs/date/sender/content where possible

Knowledge Model

Each knowledge item should support:

- Topic
- Concept
- Explanation
- Examples
- Related concepts
- Questions/open issues
- Sources
- Confidence
- Created date
- Last updated date
- Version
- Contributors

Knowledge relationships should support connections such as:

- related_to
- prerequisite_of
- example_of
- alternative_to
- contradicts
- expands_on

Daily AI Output

The AI should produce structured proposed changes similar to:

```
{
  "topic": "RAG",
  "changes": [
    {
      "type": "UPDATE",
      "section": "Why RAG is needed",
      "proposed_change": "...",
      "reason": "...",
      "source_messages": ["msg_123", "msg_127"],
      "confidence": 0.91
    }
  ],
  "questions_for_human": [],
  "related_topics": [
    "Vector Databases",
    "Embeddings",
    "LLMs"
  ]
}
```

Approval Workflow

The dashboard should show something like:

Daily Knowledge Update

- 2 new concepts
- 2 suggested improvements
- 1 possible contradiction

For each proposed change:

[Approve] [Edit] [Reject]

Also provide:

[Approve All]

Only approved changes should modify the canonical knowledge base.

Future Vision

The architecture should eventually allow ingestion from:

- WhatsApp
- Telegram
- Slack
- Meeting transcripts
- PDFs
- Articles
- YouTube transcripts
- Other documents

The long-term goal is a shared, continuously evolving "brain" for the learning community.

First Development Goal

Do NOT attempt to build the entire production system initially.

Build an MVP that can:

1. Import a WhatsApp exported chat file.
2. Parse the messages.
3. Store them.
4. Send relevant batches to an LLM.
5. Generate structured proposed knowledge updates.
6. Compare them with existing knowledge.
7. Display the proposed changes in a simple web UI.
8. Allow human approval/rejection.
9. Store approved knowledge.
10. Show relationships between knowledge items.

Before making major architectural decisions, explain the trade-offs and prefer the simplest design that keeps the system extensible.

Even better: put this into the Codex project

Create a file in the repo:

```
mindweave/  
├─ README.md  
├─ PROJECT.md  
├─ docs/  
└─ src/
```



```
| tests/  
| ...
```

Put the brief above into `PROJECT.md` . Then tell Codex:

"Read PROJECT.md. This is the product specification for MindWeave. We are going to build this incrementally. First, analyze the requirements, propose the MVP architecture and identify the first implementation milestone. Don't start coding until you've explained the architecture and trade-offs."

That gives Codex a persistent source of truth rather than relying on this conversation.

One more thing: I would keep this chat as the **product-design discussion** and Codex as the **implementation workspace**. Whenever we make an important design decision here, update `PROJECT.md` so Codex stays synchronized.



ChatGPT can make mistakes. Check important info.